

Test Strategy

Context & Scope:

Product Hunt is a content-and-community platform: browse/search products, view details, comments, upvotes, and a public GraphQL API that powers all of it.

As an end user, we can perform login, subscribe, access different features like news, forums, advertise, and new launches. Users can stay updated with the trending threads with that they can upvote and comment as well on the products.

So the strategy is about **risk-based prioritization**, not exhaustive coverage.

What matters most and Why?

- **Creating a test framework that can be executed by Devs and other QA's can also contribute to it. Also a non tech guy can also understand the code base easily.**

1. The GraphQL API is the real product surface. The web app is a client of the API — if the API is correct, most UI bugs are cosmetic; if the API is wrong, every client (web, integrations, third-party apps using the public API) inherits the bug.

- Query correctness (pagination, filtering, sorting, nested relations like post → comments → votes).

- Error handling (invalid tokens, malformed queries, rate limiting, nonexistent IDs)

- Auth boundaries (what's accessible with a dev token vs. unauthenticated)

2. For E2E and UI Testing: The flows that generate business value: discovering products (browse/search/filter), viewing product detail, and any write-path interaction (voting, commenting) if accessible with a test account. As an end user every web element should be clear enough to understand the functionality behind it, maybe with a short description while hovering or a toast message when the action is completed/UI indication. Web elements (UI) should be working and bug free making sure it is getting matched with the expected condition.

3. Data integrity across the API/UI boundary. A category of bug that's easy to miss with siloed suites: the API returns correct data, but the UI misrepresents it (stale counts, wrong pluralization, pagination that drops/duplicates items on the client).

4. CI-readiness: Tests that only work on my machine aren't tests, they're demos. Everything is written headless, with no arbitrary waits, and structured so both suites run in a single pipeline with clear pass/fail signal and artifacts (HTML report, screenshots on failure) for debugging without repro access.

5. Security posture, given the platform context: Even though this is a public content API, not a banking API, I'm treating the assessment as I would any API-first product: checking for predictable enumeration of internal IDs, verifying rate-limiting/throttling exists, confirming errors don't leak stack traces or internal schema details, and checking auth token scope.